[과제#5] 2026년 1학기 수치해석학1

***제출기한: 5월 27일(수) 오후 6시까지 LMS 업로드**

***문제: (12주차 강의자료 참고)**

1) SIR model의 무차원화 유도

학번 : 2022211414
이름 : 김민재.

1) SIR model의 무차원화 유도.

- 1-1 SIR 모델

전체 인구수 N이 변하지 않는다는 가정하고.

다음의 세 집단으로 분류.

$S(t)$: 감염 가능성이 있는 개체, susceptible.

$I(t)$: 감염된 개체, infected.

$R(t)$: 회복된 개체, removed.

β : 질병에 따른 단위 시간당 감염률.

γ : 질병에 대한 단위 시간당 회복률.

=> $1/\gamma$ 평균 감염기간, 즉 γ는 죽거나 회복까지 기간 (단위시간 등.).

- $\beta S(t)I(t)$, 단위시간 당 증가하는 감염자수. .
- $\gamma I(t)$: 단위시간 동안 감소하는 감염자 수.
  (죽거나 회복).

단위시간당.

$S(t)$의 변화율. $\frac{dS(t)}{dt} = -\beta S(t)I(t)$.

$I(t)$의 변화율. $\frac{dI(t)}{dt} = \beta S(t)I(t) - \gamma I(t)$

$R(t)$의 변화율. $\frac{dR(t)}{dt} = \gamma I(t)$.

즉. 구간 $[a,b]$, $t_i = a + i\cdot h$ $(i = 0, 1, \dots N_t)$ => $h = (b-a)/N_t$.

$S(t_{i+1}) = S(t_i) - h\beta S(t)I(t)$.

$I(t_{i+1}) = I(t_i) + h(\beta S(t_i)I(t_i) - \gamma I(t_i))$.

$R(t_{i+1}) = R(t_i) + h\cdot\gamma I(t_i)$.

- 1-2. 무차원화.

물리적 단위 제거, 비율이나 상대적인 양으로 모델을 단순화.

SIR model. 의 경우 기존변수

$S(t)$: 감염 가능성이 있는 개체, susceptible.

$I(t)$: 감염된 개체, infected.

$R(t)$: 회복된 개체, removed.

에 대하여 사람 수 N으로 나누어 정규화.

무차원화(정규화). 변수.

$u(t) = S(t)/N$

$v(t) = I(t)/N$, $u(t) + v(t) + w(t) = 1$

$w(t) = R(t)/N$

$\tau = \gamma t$. , 회복 시간이 얼마나 지났는지.

즉 평균 회복일 5일.

$\gamma = \frac{1}{5}$ ⇝ $\tau = 1$.

$t = 5$

$t = 10$ ⟶ $\tau = 2$. 즉. 평균회복 기간이 2번 지났는가?

$t = f(\tau) = \frac{\tau}{\gamma}$ 둔다면. , $f'(\tau) = \frac{1}{\gamma}$

$$U(\tau) = u(f(\tau)) = S(f(\tau))/N = \frac{1}{N}(S(f(\tau))) = \frac{1}{N}S(t)$$

$$V(\tau) = v(f(\tau)) = I(f(\tau))/N = \frac{1}{N}(I(f(\tau))) = \frac{1}{N}I(t)$$

$$W(\tau) = w(f(\tau)) = R(f(\tau))/N = \frac{1}{N}(R(f(\tau))) = \frac{1}{N}R(t).$$

$$\frac{dU(\tau)}{d\tau} = \frac{1}{N}(S'(f(\tau))\cdot f'(\tau)) = \frac{1}{N}(-\beta S(t)I(t))\frac{1}{\gamma} = -\frac{\beta}{\cancel{N}\cdot\gamma}\cancel{N}\cdot\frac{S(t)}{N}\cdot N\frac{I(t)}{N} = -\frac{\beta N}{\gamma}\cdot U(\tau)\cdot V(\tau)$$

$$\frac{dV(\tau)}{d\tau} = \frac{1}{N}(I'(f(\tau))\cdot f'(\tau)) = \frac{1}{N}(\beta S(t)I(t) - \gamma I(t))\cdot\frac{1}{\gamma} = \frac{1}{N}(N^2\beta U(\tau)V(\tau) - N\gamma V(\tau))\cdot\frac{1}{\gamma} = \frac{N\beta}{\gamma}U(\tau)\cdot V(\tau) - V(\tau).$$

$$\frac{dW(\tau)}{d\tau} = \frac{1}{N}(R'(f(\tau))\cdot f'(\tau)) = \frac{1}{N}(\gamma I(t))\frac{1}{\gamma} = \frac{1}{\cancel{N}}\cancel{N}\frac{I(t)}{N} = V(\tau).$$

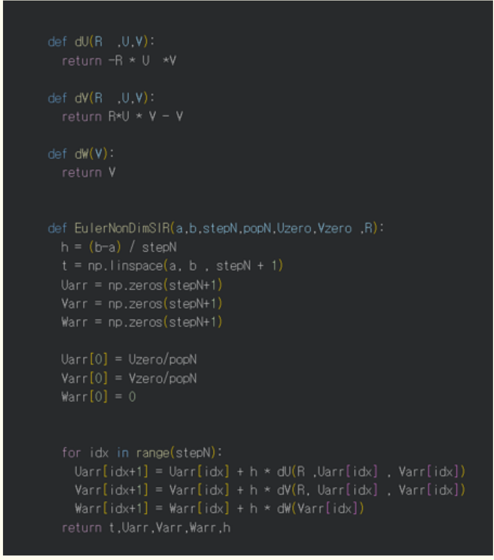
$\frac{\beta N}{\gamma} = R_0$, 기초감염 재생산수로 두면.

SIR model 의 무차원 화 유도식은 다음과 같다.

$$\frac{dU(\tau)}{d\tau} = -R_0 \cdot U(\tau) \cdot V(\tau)$$

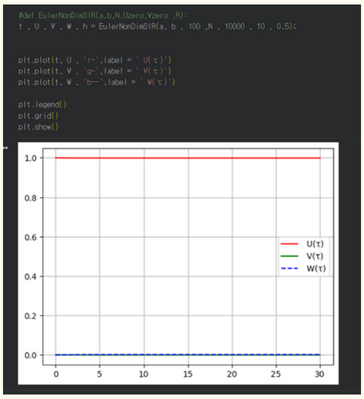
$$\frac{dV(\tau)}{d\tau} = R_0 U(\tau) \cdot V(\tau) - V(\tau)$$

$$\frac{dW(\tau)}{d\tau} = V(\tau)$$



```
def dU(R  ,U,V):
  return -R * U  *V

def dV(R  ,U,V):
  return R*U * V - V

def dW(V):
  return V


def EulerNonDimSIR(a,b,stepN,popN,Uzero,Vzero ,R):
  h = (b-a) / stepN
  t = np.linspace(a, b , stepN + 1)
  Uarr = np.zeros(stepN+1)
  Varr = np.zeros(stepN+1)
  Warr = np.zeros(stepN+1)

  Uarr[0] = Uzero/popN
  Varr[0] = Vzero/popN
  Warr[0] = 0


  for idx in range(stepN):
    Uarr[idx+1] = Uarr[idx] + h * dU(R ,Uarr[idx] , Varr[idx])
    Varr[idx+1] = Varr[idx] + h * dV(R, Uarr[idx] , Varr[idx])
    Warr[idx+1] = Warr[idx] + h * dW(Varr[idx])
  return t,Uarr,Varr,Warr,h
```

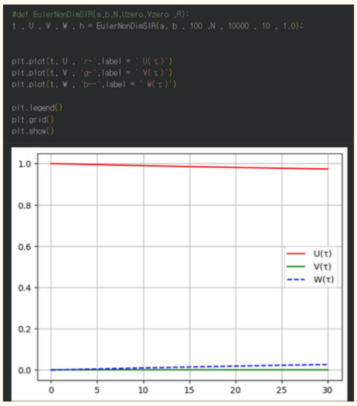
2) 무차원화 SIR 모델의 R_0의 효과를 확인하세요. 다음 그래프와 같아지도록 하는 코드와 변수를 제출하면 됩니다.



```
#def EulerNonDimSIR(a,b,N,Uzero,Vzero ,R):
t , U , V , W , h = EulerNonDimSIR(a, b , 100 ,N , 10000 , 10 , 0.5);


plt.plot(t, U , 'r-',label = ' U(τ)')
plt.plot(t, V , 'g-',label = ' V(τ)')
plt.plot(t, W , 'b--',label = ' W(τ)')

plt.legend()
plt.grid()
plt.show()
```

1.0
0.8
0.6
0.4
0.2
0.0
0
5
10
15
20
25
30
U(τ)
V(τ)
W(τ)

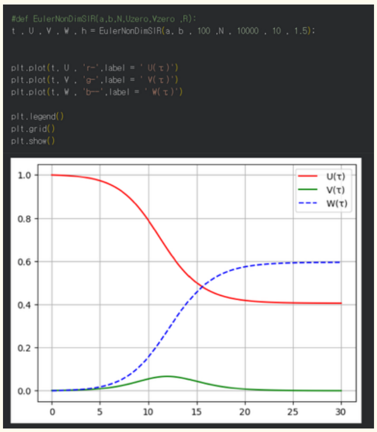
$R_0 = 0.5$



```
#def EulerNonDimSIR(a,b,N,Uzero,Vzero ,R):
t , U , V , W , h = EulerNonDimSIR(a, b , 100 ,N , 10000 , 10 , 1.0);


plt.plot(t, U , 'r-',label = ' U(τ)')
plt.plot(t, V , 'g-',label = ' V(τ)')
plt.plot(t, W , 'b--',label = ' W(τ)')

plt.legend()
plt.grid()
plt.show()
```

1.0
0.8
0.6
0.4
0.2
0.0
0
5
10
15
20
25
30
U(τ)
V(τ)
W(τ)

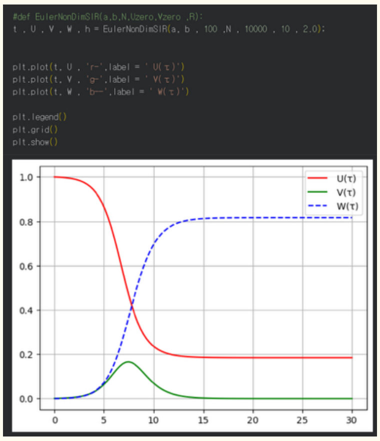
$R_0 = 1.0$



```
#def EulerNonDimSIR(a,b,N,Uzero,Vzero ,R):
t , U , V , W , h = EulerNonDimSIR(a, b , 100 ,N , 10000 , 10 , 1.5);


plt.plot(t, U , 'r-',label = ' U(τ)')
plt.plot(t, V , 'g-',label = ' V(τ)')
plt.plot(t, W , 'b--',label = ' W(τ)')

plt.legend()
plt.grid()
plt.show()
```

1.0
0.8
0.6
0.4
0.2
0.0
0
5
10
15
20
25
30
U(τ)
V(τ)
W(τ)

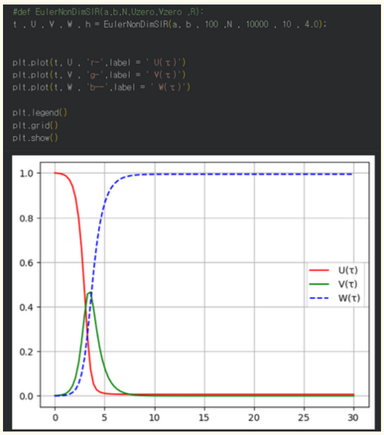
$R_0 = 1.5$



```
#def EulerNonDimSIR(a,b,N,Uzero,Vzero ,R):
t , U , V , W , h = EulerNonDimSIR(a, b , 100 ,N , 10000 , 10 , 2.0);


plt.plot(t, U , 'r-',label = ' U(τ)')
plt.plot(t, V , 'g-',label = ' V(τ)')
plt.plot(t, W , 'b--',label = ' W(τ)')

plt.legend()
plt.grid()
plt.show()
```

1.0
0.8
0.6
0.4
0.2
0.0
0
5
10
15
20
25
30
U(τ)
V(τ)
W(τ)

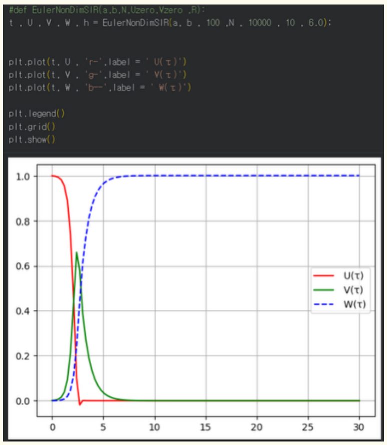
$R_0 = 2.0$



```
#def EulerNonDimSIR(a,b,N,Uzero,Vzero ,R):
t , U , V , W , h = EulerNonDimSIR(a, b , 100 ,N , 10000 , 10 , 4.0);


plt.plot(t, U , 'r-',label = ' U(τ)')
plt.plot(t, V , 'g-',label = ' V(τ)')
plt.plot(t, W , 'b--',label = ' W(τ)')

plt.legend()
plt.grid()
plt.show()
```

1.0
0.8
0.6
0.4
0.2
0.0
0
5
10
15
20
25
30
U(τ)
V(τ)
W(τ)

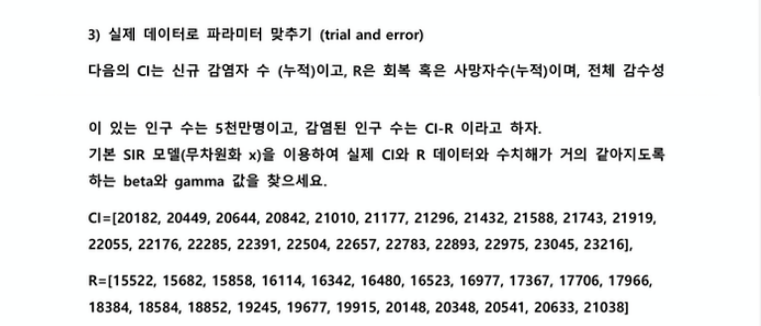
$R_0 = 4.0$



```
#def EulerNonDimSIR(a,b,N,Uzero,Vzero ,R):
t , U , V , W , h = EulerNonDimSIR(a, b , 100 ,N , 10000 , 10 , 6.0);


plt.plot(t, U , 'r-',label = ' U(τ)')
plt.plot(t, V , 'g-',label = ' V(τ)')
plt.plot(t, W , 'b--',label = ' W(τ)')

plt.legend()
plt.grid()
plt.show()
```

1.0
0.8
0.6
0.4
0.2
0.0
0
5
10
15
20
25
30
U(τ)
V(τ)
W(τ)

$R_0 = 6.0$



3) 실제 데이터로 파라미터 맞추기 (trial and error)

다음의 CI는 신규 감염자 수 (누적)이고, R은 회복 혹은 사망자수(누적)이며, 전체 감수성

이 있는 인구 수는 5천만명이고, 감염된 인구 수는 CI-R 이라고 하자.
기본 SIR 모델(무차원화 x)을 이용하여 실제 CI와 R 데이터와 수치해가 거의 같아지도록 하는 beta와 gamma 값을 찾으세요.

CI=[20182, 20449, 20644, 20842, 21010, 21177, 21296, 21432, 21588, 21743, 21919, 22055, 22176, 22285, 22391, 22504, 22657, 22783, 22893, 22975, 23045, 23216],

R=[15522, 15682, 15858, 16114, 16342, 16480, 16523, 16977, 17367, 17706, 17966, 18384, 18584, 18852, 19245, 19677, 19915, 20148, 20348, 20541, 20633, 21038]

S(t)의 변화율. $\frac{dS(t)}{dt} = -\beta S(t) I(t).$

I(t)의 변화율. $\frac{dI(t)}{dt} = \beta S(t) I(t) - r I(t)$

R(t)의 변화율. $\frac{dR(t)}{dt} = r I(t).$

$$\begin{tikzcd} S(t) \arrow[r, bend left, "\uparrow"'] & I(t) \arrow[r, bend left, "\uparrow"'] & R(t) \end{tikzcd}$$

$\pm$ → 신규감염자수.

(사망자수 or 회복자수.)

$$Cl_n - Cl_0 = \int_0^n \beta S(t) I(t)\, dt = -\int_0^n -\beta S(t) I(t)\, dt = -\Big[S(t) \Big]_0^n = -\big(S(n) - S(0) \big) = S(0) - S(n).$$

$$R_n - R_0 = \int_0^n r I(t)\, dt = \int_0^n r I(t)\, dt = \Big[R(t) \Big]_0^n = R(n) - R(0).$$

$$\begin{matrix} Cl_n - Cl_0 \\ -(R_n - R_0) \end{matrix} = \int_0^n \beta S(t) I(t) - r I(t)\, dt = \Big[I(t) \Big]_0^n = I(n) - I(0).$$

$$Cl_n - R_n - (Cl_0 - R_0) = I(n) - I(0)$$

$$Cl_n - R_n = I(n)$$

$$Cl_{n+1} - Cl_n = \begin{matrix} -(S(n+1) - S(0)) \\ - \\ -(S(n) - S(0)) \end{matrix} = -(S(n+1) - S(n)) = S(n) - S(n+1).$$

$$\left. \begin{aligned} Cl_{n+1} - Cl_n &= S(n) - S(n+1) \\ S(n+1) &= S(n) - Cl_n + Cl_{n+1} \\ &= S(n) - h \cdot \beta \cdot S(n)\, I(n) \end{aligned} \right\} \Rightarrow$$

$h = 1$

$$\underline{\beta S(n) I(n) = -Cl_n + Cl_{n+1}.}$$

$$\beta = \frac{Cl_{n+1} - Cl_n}{S(n) \cdot I(n)}$$

$$\left. \begin{aligned} R_{n+1} - R_n &= R(n+1) - R(n) \\ R(n+1) &= R(n) + R_{n+1} - R_n \\ &= R(n) + h\, r\, I(n) \end{aligned} \right\} \rightsquigarrow$$

$n = 1.$

$$r I(n) = \underline{R_{n+1}} - R_n.$$

$$r = \frac{R_{n+1} - R_n}{I(n)}$$

$$Cl_{n+1} - R_{n+1} - (Cl_n - R_n) = I(n+1) - I(n).$$

$$I(n) + Cl_{n+1} - R_{n+1} - (Cl_n - R_n) = I(n+1).$$

$$I(n) + \underline{h(\beta S(n) I(n) - r I(n))} = I(n+1).$$

$\hookrightarrow$ $h = 1$

$$\beta S(n) I(n) - r I(n) = Cl_{n+1} - R_{n+1} - (Cl_n - R_n).$$

$$Cl_n - Cl_0 = -S(n) + S(0).$$

$$R_n - R_0 = R(n) - R(0).$$

$$Cl_n - R_n = I(n)$$

$$S(n) = S(0) - Cl_n.$$

$$R(n) = R_n + R(0).$$

$$I(n) = Cl_n - R_n + I(0).$$

$$= -S(n) + S(0) - R(n) + R(0) + I(0)$$

$$= 50000000 - S(n) - R(n).$$

$S(0) = 50{,}000{,}000.$ 초기조건.

$I(0) = Cl_0 - R_0 = 4660.$

$$\beta = \frac{Cl_{n+1} - Cl_n}{S(n) \cdot I(n)} = \frac{Cl_{n+1} - Cl_n}{(S(0) - Cl_n)(Cl_n - R_n)}.$$

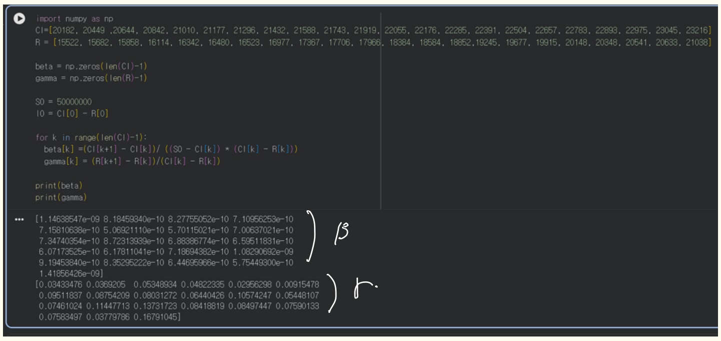
$$r = \frac{R_{n+1} - R_n}{I(n)} = \frac{R_{n+1} - R_n}{Cl_n - R_n}$$

$\beta_0 \cdots\cdots \beta_{n-1} \longrightarrow \beta_{avg}$

기-출계.

평균값 사용.

$r_0 \cdots r_{n-1} \longrightarrow r_{avg}.$

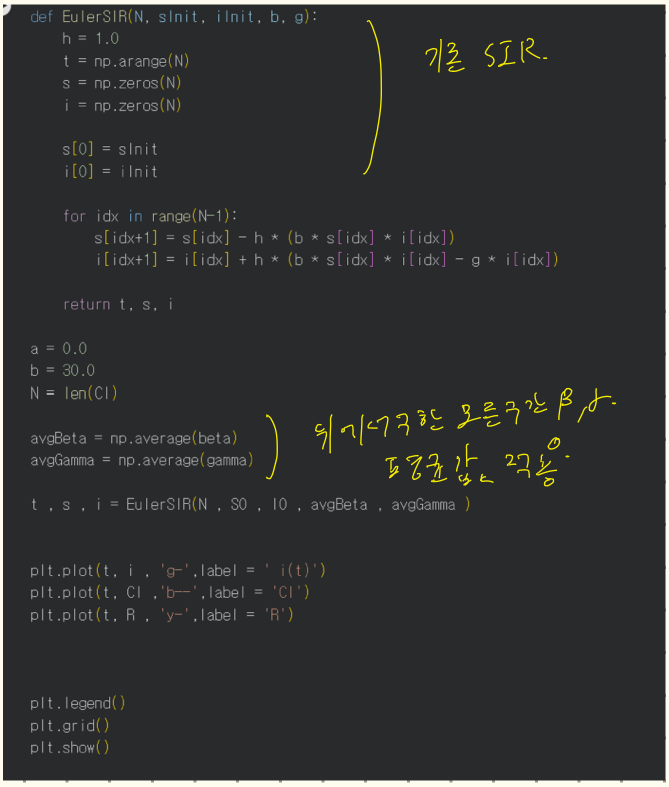


```
import numpy as np
CI=[20182, 20449 ,20644, 20842, 21010, 21177, 21296, 21432, 21588, 21743, 21919, 22055, 22176, 22285, 22391, 22504, 22657, 22783, 22893, 22975, 23045, 23216]
R = [15522, 15682, 15858, 16114, 16342, 16480, 16523, 16977, 17367, 17706, 17966, 18384, 18584, 18852,19245, 19677, 19915, 20148, 20348, 20541, 20633, 21038]

beta = np.zeros(len(CI)-1)
gamma = np.zeros(len(R)-1)

S0 = 50000000
I0 = CI[0] - R[0]

for k in range(len(CI)-1):
  beta[k] =(CI[k+1] - CI[k])/ ((S0 - CI[k]) * (CI[k] - R[k]))
  gamma[k] = (R[k+1] - R[k])/(CI[k] - R[k])

print(beta)
print(gamma)
```

```
[1.14638547e-09 8.18459340e-10 8.27755052e-10 7.10956253e-10
 7.15810638e-10 5.06921110e-10 5.70115021e-10 7.00637021e-10
 7.34740354e-10 8.72313939e-10 6.88386774e-10 6.59511831e-10
 6.07173525e-10 6.17811041e-10 7.18694382e-10 1.08290692e-09
 9.19453840e-10 8.35295222e-10 6.44695966e-10 5.75449300e-10
 1.41856426e-09]
[0.03433476 0.0369205  0.05348934 0.04822335 0.02956298 0.00915478
 0.09511837 0.08754209 0.08031272 0.06440426 0.10574247 0.05448107
 0.07461024 0.11447713 0.13731723 0.08418819 0.08497447 0.07590133
 0.07583497 0.03779786 0.16791045]
```

β

γ.

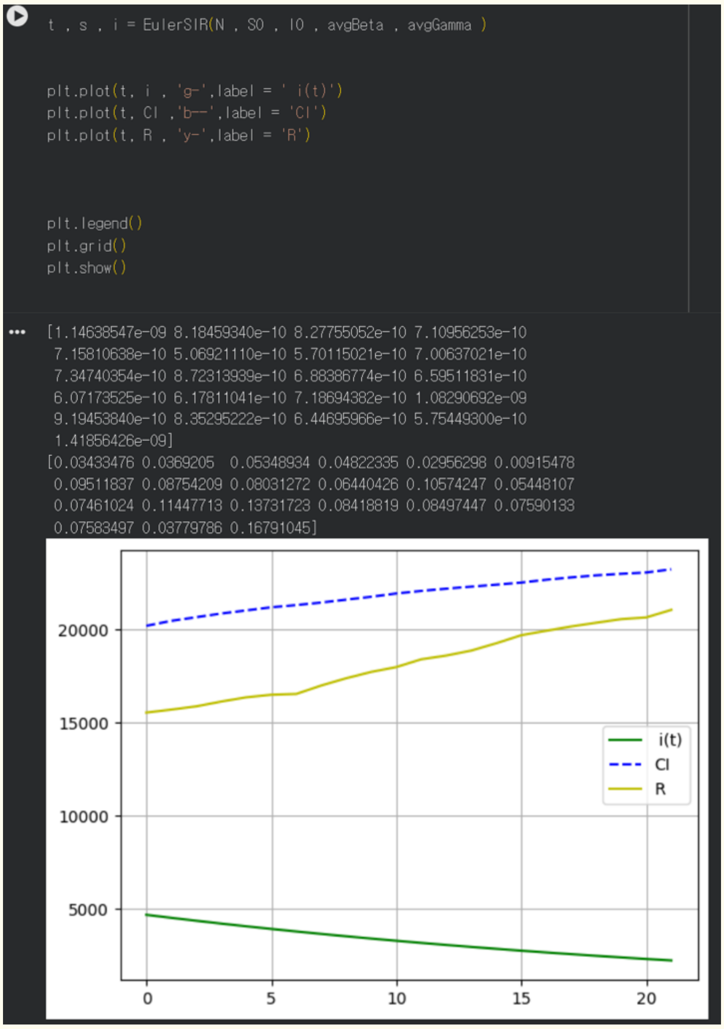


```
def EulerSIR(N, sInit, iInit, b, g):
    h = 1.0
    t = np.arange(N)
    s = np.zeros(N)
    i = np.zeros(N)

    s[0] = sInit
    i[0] = iInit

    for idx in range(N-1):
        s[idx+1] = s[idx] - h * (b * s[idx] * i[idx])
        i[idx+1] = i[idx] + h * (b * s[idx] * i[idx] - g * i[idx])

    return t, s, i

a = 0.0
b = 30.0
N = len(CI)

avgBeta = np.average(beta)
avgGamma = np.average(gamma)

t , s , i = EulerSIR(N , S0 , I0 , avgBeta , avgGamma )


plt.plot(t, i , 'g-',label = ' i(t)')
plt.plot(t, CI ,'b--',label = 'CI')
plt.plot(t, R , 'y-',label = 'R')



plt.legend()
plt.grid()
plt.show()
```

기존 SIR.

위에서 구한 모든 구간 β, γ.
평균값 적용.

결과 =>



```
t , s , i = EulerSIR(N , S0 , I0 , avgBeta , avgGamma )


plt.plot(t, i , 'g-',label = ' i(t)')
plt.plot(t, CI ,'b--',label = 'CI')
plt.plot(t, R , 'y-',label = 'R')


plt.legend()
plt.grid()
plt.show()
```

```
[1.14638547e-09 8.18459340e-10 8.27755052e-10 7.10956253e-10
 7.15810638e-10 5.06921110e-10 5.70115021e-10 7.00637021e-10
 7.34740354e-10 8.72313939e-10 6.88386774e-10 6.59511831e-10
 6.07173525e-10 6.17811041e-10 7.18694382e-10 1.08290692e-09
 9.19453840e-10 8.35295222e-10 6.44695966e-10 5.75449300e-10
 1.41856426e-09]
[0.03433476 0.0369205  0.05348934 0.04822335 0.02956298 0.00915478
 0.09511837 0.08754209 0.08031272 0.06440426 0.10574247 0.05448107
 0.07461024 0.11447713 0.13731723 0.08418819 0.08497447 0.07590133
 0.07583497 0.03779786 0.16791045]
```